# LAB6 결과 보고서

*STM32_2 컴퓨터와 FPGA 통신 — FPGA 서보모터·SRAM·DAC*


과목 전자전기컴퓨터설계실험 2

실험일 2026년 10월 26일

성명 김태이

학번 2023440158


GitHub github.com/tae2yi/UOS_ece2/tree/95b1fc7/lab6

시연 영상 github.com/tae2yi/UOS_ece2/tree/master/lab6/report/final/videos

# 1 실험 범위와 결과 요약

10월 26일 실험에서 NUCLEO-G474RE로 STM32_2 과제 세 개(PC와 MCU 입출력, MCU에서 FPGA로 보낸 값 확인, UART 명령으로 FPGA LED 제어)를, Combo II-DLD로 교재 실습 세 개(서보모터, SRAM, DAC)를 동작시키고 영상 6개로 남겼다. 이 보고서는 영상에서 읽은 값을 예비보고서의 예상과 비교한다. 보드와 화면 사진은 모두 영상의 장면이다.

| 실험 | 영상에서 확인한 것 | 예상과 비교 |
|---|---|---|
| 과제5 | '1' 'a' 'A' → 0x31 0x61 0x41 | 일치 |
| 과제6 | MCU 결과 0, FPGA 포트에 1 a A | 일치 |
| 과제7 | '0' 유지, '1' 켬, '0' 끔, LF 유지 | 일치 |
| 서보 | 45° ↔ 90° ↔ 135°, 한 번에 약 9° | 1~2 ms 범위·단계 일치 |
| SRAM | 읽기 때만 LED5~8에 0011, 0010 | 버스 전환 동작 일치 |
| DAC | 초당 약 100 증가, 1.27 s마다 MSB 반전 | 10 ms 갱신, 2.56 s 주기 일치 |

여섯 실험 모두 예비보고서가 예상한 동작이 나왔다. 서보는 핀을 확정한 뒤 매뉴얼 코드가 50 MHz 타이밍을 −0.848 ns 위반해, 예비보고서에서 준비한 대비안(나눗셈 제거)의 bitstream을 썼다. DAC 전압은 보드의 전압계 막대가 값 약 50에서 가득 차 최대 전압을 읽지 못했다. 정량 비교는 영상의 프레임 간격(0.033 s)과 눈금 읽기 정밀도 안에서 한 것이다.

# 2 실험 환경과 측정 방법

| 구분 | 사용한 것 |
|---|---|
| MCU 보드 | NUCLEO-G474RE, 직렬 모니터 9600 8N1 |
| FPGA 보드 | Combo II-DLD, xc7s75fgga484-1, 클럭 50 MHz |
| MCU 코드 | ece2_474 v1.1.0 + ece2_474_routine.c (STM32_2_TASK 5, 6) |
| FPGA 설계 | servo_board_mul, sram, dac, mcu_uart_led |
| 배선 | PC10 → BDIO_0(F8), GND → J6, DIP SW5(BD_SEL) 켬 |
| 기록 | 휴대전화 영상 6개 (30 fps, 음성 포함) |

과제6·7에서는 MCU의 ST-LINK 포트와 FPGA의 USB-UART 포트를 서로 다른 PC의 직렬 모니터로 열었다. 두 보드는 점퍼선 두 개(신호와 GND)로만 이어지므로, 신호의 기준이 되는 GND만 공통이면 PC가 달라도 된다.

### 영상에서 값을 읽은 방법

- DAC·과제7: 프레임마다 LED 위치의 밝은 빨간 화소 수로 켜짐을 판정했다. 휴대전화가 흔들리므로 기준 프레임과의 이동량을 위상 상관으로 구해 LED 위치를 보정했다 (한 프레임 0.033 s).
- 서보: 버튼은 영상 음성의 클릭음으로 찾고, 팔 각도는 보드에 인쇄된 0~180 눈금을 장면에서 읽었다(약 ±5°).
- SRAM LED 값과 직렬 모니터 글자는 장면을 확대해 사람이 읽었다.

측정 코드:
https://github.com/tae2yi/UOS_ece2/blob/95b1fc7/lab6/report/final/make_figures.py

측정값:
https://github.com/tae2yi/UOS_ece2/tree/95b1fc7/lab6/report/final/data

# 3 STM32_2 과제5 PC와 MCU 입출력

### 코드

getchar가 UART2(ST-LINK VCP)에서 한 바이트를 받을 때까지 기다리고, 받은 바이트를 print_byte로 글자와 16진수로 출력한다. 과제5와 과제6·7은 STM32_2_TASK 매크로로 나뉜다[1].

```
void polling_routine(void)
{
  /* PC에서 문자가 올 때까지 getchar 안에서 기다립니다. */
  int ch = getchar();
  if (ch == EOF)
  {
    return;
  }
  uint8_t data = (uint8_t)ch;
#if STM32_2_TASK == 5
  printf("받은 문자: ");
  print_byte(data);
  printf("\r\n");
#else
  HAL_StatusTypeDef result =
    HAL_UART_Transmit(&huart3, &data, 1U, 100U);
  printf("FPGA로 보냄: ");
  print_byte(data);
  printf(", 전송 결과: %d\r\n", (int)result);
#endif
}
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/95b1fc7/lab6/stm32_lab/Core/Src/ece2_474_routine.c#L85-L105

### 보드 관찰

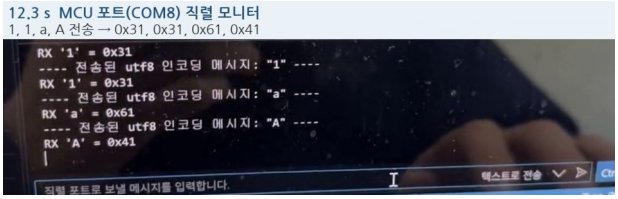



그림 1. 과제5 직렬 모니터 화면. 입력 조건: COM8, 9600, 줄 끝 없음으로 1, 1, a, A를 차례로 전송. 확인한 동작: 문자마다 받은 글자와 16진수 값이 한 줄씩 출력된다.

| 보낸 문자 | 화면의 값 | 예상 |
|---|---|---|
| '1' | 0x31 | 0x31 |
| '1' | 0x31 | 0x31 |
| 'a' | 0x61 | 0x61 |
| 'A' | 0x41 | 0x41 |

리셋 직후 Hello, World!가 한 번 나온 뒤, 문자를 보낼 때마다 한 줄씩 응답이 왔다. 줄 끝 설정이 없음이라 CR·LF가 함께 가지 않아 문자 하나에 한 줄만 나왔다. 응답이 없던 동안은 getchar가 다음 문자를 기다리는 상태였고, 문자를 보내면 바로 응답했으므로 멈춘 것이 아니다.

### 예상과의 비교

'1' 'a' 'A'가 0x31, 0x61, 0x41로 나와 예비보고서의 표와 같다. 'A'와 'a'의 차이는 0x20(비트 5)이다. 줄 끝 CR LF로 바꿨을 때 0x0D, 0x0A가 더 나오는지는 과제6 영상에서 LF로 확인했다(4장). 한글 입력과 디버거 일시 정지는 이번 기록에 없다.

## 4 STM32_2 과제6 MCU에서 FPGA로 보낸 값

### 설계

MCU는 받은 바이트를 HAL_UART_Transmit으로 UART3(PC10)에 보내고 결과를 출력한다. FPGA의 mcu_uart_led는 BDIO_0(F8)에서 받은 바이트를 USB-UART(F6)로 그대로 PC에 보내고, LED2~8에 하위 7비트를 표시한다. 50 MHz를 5208로 나눠 9600 bps를 만든다[2].

```
always @(posedge clk_50mhz) begin
  if (rst_p) begin
    tx_valid <= 0; tx_data <= 0; led_cmd <= 0; led_byte <= 0;
  end else begin
    tx_valid <= 0;
    if (rx_valid) begin
      led_byte <= rx_data[6:0];
      if (tx_ready) begin
        tx_data <= rx_data;
        tx_valid <= 1;
      end
      case (rx_data)
        8'h30: led_cmd <= 1'b0;      // '0'
        8'h31: led_cmd <= 1'b1;      // '1'
        8'h32: led_cmd <= ~led_cmd;  // '2'
        default: led_cmd <= led_cmd;
      endcase
    end
  end
end
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/95b1fc7/lab6/lab6_04_uart_led/src/mcu_uart_led.v#L33-L52

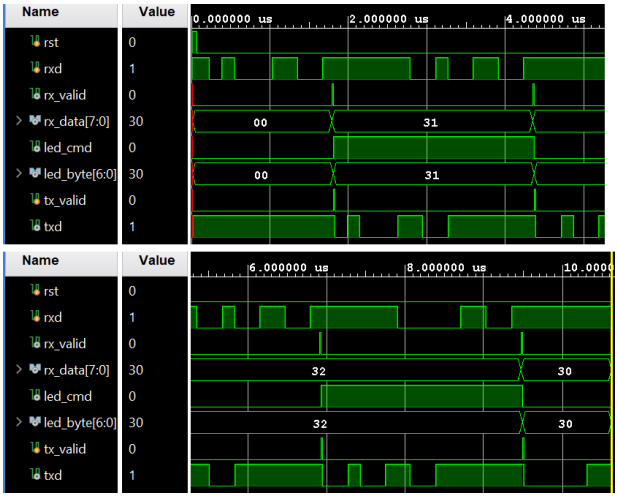



그림 2. 과제6·7 FPGA Vivado 파형(예비, 한 비트 8클록). rxd로 '1'·'2'·'2'·'0'이 들어오면 led_cmd가 1·0·1·0이 되고 같은 바이트가 txd로 나간다.

### 보드 관찰




그림 3. 과제6 두 직렬 모니터. 입력 조건: MCU 포트(COM8, 줄 끝 LF)에 1, a, A 전송. 확인한 동작: 왼쪽 MCU 화면에 보낸 문자·LF와 결과 0, 오른쪽 FPGA 포트 화면에 1, a, A가 한 줄씩 들어온다.

| 보낸 것 | MCU 출력 | FPGA 포트 |
|---|---|---|
| '1' LF | 0x31, 0x0A, 결과 0 | 1 (줄바꿈) |
| 'a' LF | 0x61, 0x0A, 결과 0 | a (줄바꿈) |
| 'A' LF | 0x41, 0x0A, 결과 0 | A (줄바꿈) |

직렬 모니터의 줄 끝을 LF로 두어 문자마다 LF(0x0A)가 함께 갔고, MCU는 두 바이트를 각각 UART3로 보냈다. FPGA는 LF까지 그대로 돌려보냈기 때문에 FPGA 쪽 화면에는 문자가 한 줄에 하나씩 찍혔다. 보낸 세 문자가 빠짐없이 같은 순서로 도착했다.

### 예상과의 비교

예비보고서는 「MCU: 전송 결과 0 / FPGA 포트: 1」, 「'A' 'a'가 FPGA 포트에 표시」를 예상했고 그대로 나왔다. 결과 0(HAL_OK)은 MCU가 송신을 마쳤다는 뜻일 뿐이라, 실제 전달은 FPGA 포트 화면으로 확인했다. 시뮬레이션에서 MCU 호스트 출력 파일을 FPGA 테스트벤치에 넣어 본 3바이트 에코와 같은 결과다. GND 선이나 점퍼선을 뺀 비교 실험은 하지 않았다.

# 5 STM32_2 과제7 UART 명령으로 LED

MCU 코드와 FPGA 설계는 과제6과 같다. FPGA는 받은 바이트가 '0'(0x30)이면 LED1을 끄고, '1'(0x31)이면 켜고, '2'(0x32)이면 뒤집고, 그 밖의 바이트에서는 그대로 둔다. LED2~8에는 마지막에 받은 바이트가 계속 표시된다.

## 보드 관찰

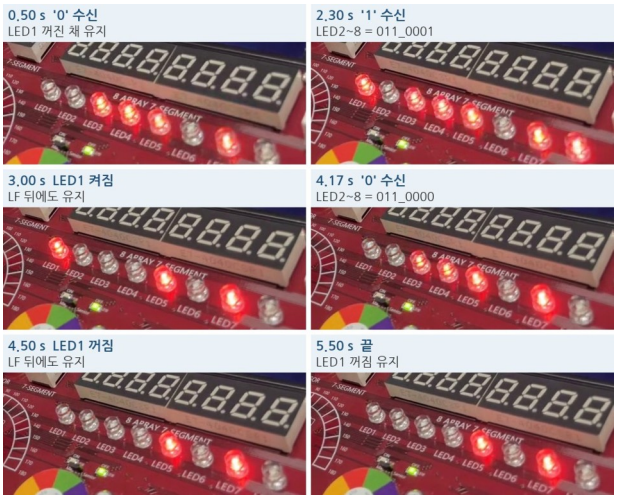



그림 4. 과제7 보드 사진. 입력 조건: MCU 포트(줄 끝 LF)에 0, 1, 0 전송. 확인한 동작: 문자가 도착한 프레임에 LED2~8이 그 문자를 보이고, LED1은 '1'에서 켜지고 '0'에서 꺼진다.

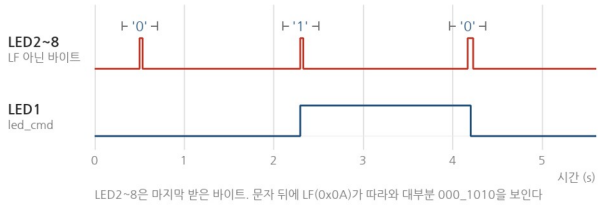



그림 5. 과제7 측정. 위는 LED2~8이 LF가 아닌 바이트를 보인 프레임, 아래는 LED1. 둘 다 프레임마다 LED 화소로 읽었다.

| 시각 (s) | LED2~8 | 받은 문자 | LED1 |
|---|---|---|---|
| 0.50 | 011_1010 | '0' (+LF) | 꺼짐 유지 |
| 2.30 | 011_1011 | '1' (+LF) | 켜짐 |
| 4.17~4.20 | 011_1010 | '0' (+LF) | 4.20 s 꺼짐 |
| 그 밖 | 000_1010 | LF | 유지 |

측정값:
https://github.com/tae2yi/UOS_ece2/blob/95b1fc7/lab6/report/final/data/stm32_task7_led.csv

문자가 도착한 프레임의 LED2~8은 '1'(011_0001) 또는 '0'(011_0000)에 LF(000_1010)의 비트가 겹친 모양이다. 한 프레임(33 ms) 안에 문자와 LF가 차례로 도착해 두 값이 모두 찍힌 것이다. MCU는 문자를 FPGA로 보낸 뒤 결과 한 줄을 PC에 출력하고 나서 다음 바이트(LF)를 읽으므로, 문자 표시는 1~2프레임만 남았다.

## 예상과의 비교

'1'에서 켜지고 '0'에서 꺼졌으며, 이미 꺼진 상태에서 받은 '0'은 아무 변화가 없었다. 문자마다 따라온 LF는 그 밖의 문자로 처리되어 LED1을 바꾸지 않았다. 예비보고서의 「줄바꿈 CR LF: LED 변화 없음」과 같은 동작이다. 반전 명령 '2'와 'x' 같은 다른 문자는 이번 영상에 없고, 시뮬레이션(명령 9개 일치)으로만 확인했다.

# 6 FPGA-01 서보모터

## 설계와 실습 전 수정

서보는 20 ms마다 오는 펄스의 HIGH 폭으로 각도를 정한다[2]. servo_position이 왼쪽·오른쪽 버튼 펄스마다 폭을 100 µs씩 줄이거나 늘리고(1000~2000 µs, 리셋 1500 µs), servo_pwm이 50 MHz를 세어 펄스를 만든다. 버튼은 20 ms 동안 같은 값일 때만 인정하는 button_pulse를 거쳐 한 번 누름에 한 펄스가 된다.

교재 서보 실습 핀 표로 SERVO = AA22를 정하고 다시 구현하자 매뉴얼 코드가 WNS −0.848 ns로 타이밍을 위반했다. 예비보고서에서 예상한 대로 펄스 폭 계산(32비트 곱셈과 1000 나눗셈)이 한 클록 안에 있는 경로였다. 그래서 1 µs당 클록 수(50)를 곱하기만 하는 대비안으로 bitstream을 만들어 보드에 올렸다.

```
width_cycles<=(CLK_HZ%1_000_000==0)
  ? pulse_us*(CLK_HZ/1_000_000)
  : pulse_us*(CLK_HZ/1000)/1000;
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/95b1fc7/lab6/lab6_01_servo/variants/servo_pwm_mul.v#L20-L22

| 항목 | 매뉴얼 코드 | 대비안 (사용) |
|---|---|---|
| WNS (50 MHz, AA22) | −0.848 ns | +13.183 ns |
| LUT · 레지스터 | 466 · 121 | 115 · 127 |
| bitstream | servo_board.bit | servo_board_mul.bit |

리포트:
https://github.com/tae2yi/UOS_ece2/blob/95b1fc7/lab6/report/pre/logs/lab6_01_servo_mul_timing.rpt

### 보드 관찰




그림 6. FPGA-01 보드 사진. 입력 조건: SW2(왼쪽)·SW3(오른쪽)을 약 0.17 s 간격으로 연달아 누름. 확인한 동작: 팔이 가운데에서 양 끝까지 움직이고 끝에서 멈춘다. 각도는 보드 눈금을 읽은 값(90° = 위쪽).

| 구간 (s) | 클릭음 | 팔 각도 | 설계상 펄스 폭 |
|---|---|---|---|
| 0.1 | — | 약 90° | 1.5 ms (리셋) |
| 0.24~0.88 | 5번 | 약 45° (1.2 s) | 1.0 ms (왼쪽 끝) |
| 1.04~1.77 | 5번 | 약 90° (2.0 s) | 1.5 ms |
| 1.94~2.64 | 5번 | 약 135° (3.0 s) | 2.0 ms (오른쪽 끝) |
| 2.79~3.8 | 2번+ | 약 90° (4.0 s) | 1.5 ms |
| 4.49~4.82 | 3번 | 약 63° (5.1 s) | 1.2 ms |

측정값:
https://github.com/tae2yi/UOS_ece2/blob/95b1fc7/lab6/report/final/data/servo_angle_readout.csv

클릭음 다섯 번마다 팔이 가운데와 끝 사이를 오갔다. 양 끝 45°와 135°의 차이 90°를 10단계로 나누면 한 번에 약 9°이고, 마지막 세 번 누름 뒤 90 − 3 × 9 = 63°와 읽은 값이 맞는다. 2.79~3.8 s에는 손 위치가 바뀌어 클릭음이 두 번만 잡혔지만 팔이 가운데로 돌아왔으므로 다섯 번 누른 것으로 본다.

### 예상과의 비교

예비보고서는 「리셋 뒤 가운데」, 「한 번에 한 단계(0.1 ms)만 회전」, 「끝에서 멈춤」을 예상했다. 0.17 s 간격으로 빠르게 눌러도 누름마다 한 단계씩만 움직였고(20 ms 떨림 제거가 짧은 누름을 놓치지 않음), 끝 위치 45°·135°가 1 ms·2 ms에 해당한다. 오실로스코프로 20 ms 주기와 HIGH 폭을 재지는 않아, 펄스 폭은 각도로 간접 확인한 것이다.

# 7 FPGA-02 SRAM

### 설계와 핀

sram_bus_control이 버튼 두 개로 /CS·/OE·/WE를 만들고, FPGA는 쓰기 동안에만 데이터 버스를 구동하며 나머지는 Z로 둔다[3]. DIP 1~4가 주소, 5~8이 데이터이고 SW2가 쓰기, SW3이 읽기다. 읽는 동안만 LED5~8에 ram_data[3:0]이 나온다.

```
assign ram_address={14'b0,address};
assign ram_data=drive_data ? {12'b0,data_in} : 16'bz;
assign led_out=read_data ? {4'b0,ram_data[3:0]} : 8'b0;
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/95b1fc7/lab6/lab6_02_sram/src/sram.v#L12-L14

실험 전에 교재 p.603 핀 표로 주소 18개, 데이터 16개, /CS·/WE·/OE 핀을 채우고 bitstream을 만들었다. /UB·/LB는 회로도에서 GND에 묶여 있어 설계에 출력을 더하지 않았다. 조합 회로라 타이밍 경로는 없다(LUT 4개).

XDC:
https://github.com/tae2yi/UOS_ece2/blob/95b1fc7/lab6/lab6_02_sram/constraints/sram.xdc

### 보드 관찰

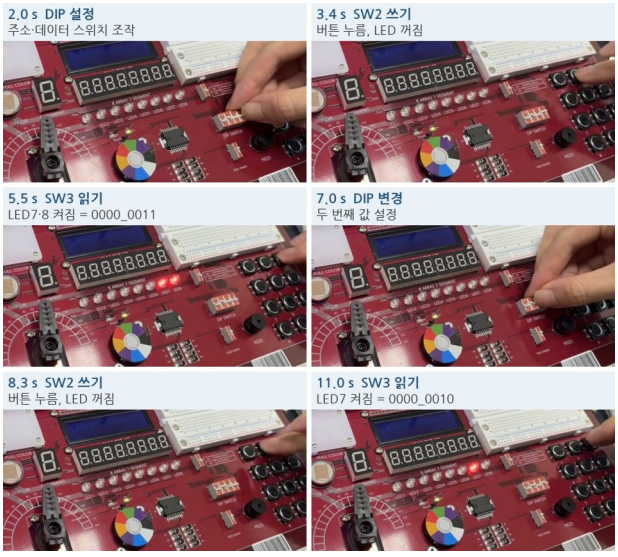



그림 7. FPGA-02 보드 사진. 입력 조건: DIP로 주소·데이터 설정 → SW2 쓰기 → DIP 변경 → SW3 읽기, 두 번 반복. 확인한 동작: 쓰는 동안 LED는 꺼져 있고, 읽는 동안만 저장된 값이 LED5~8에 나온다.

| 시각 (s) | LED1~8 | 값 | 동작 |
|---|---|---|---|
| 5.5 | 0000_0011 | 3 | 첫 번째 읽기 |
| 11.0 | 0000_0010 | 2 | 두 번째 읽기 |

측정값:
https://github.com/tae2yi/UOS_ece2/blob/95b1fc7/lab6/report/final/data/sram_led_readout.csv

두 번 모두 쓰기를 한 뒤 DIP 스위치를 바꾸고 나서 읽었는데, LED에는 바뀐 스위치 값이 아니라 저장된 값(3, 2)이 나왔다. 읽을 때 데이터 버스를 SRAM이 구동하고 FPGA는 Z로 물러나 있다는 뜻이다. 버튼을 떼면 LED가 바로 꺼졌다.

### 예상과의 비교

예비보고서의 「SW3을 누르는 동안 LED = 쓴 값」, 「쓰기 중 LED 꺼짐」과 같다. 상위 LED1~4는 설계상 항상 0이고 실제로도 꺼져 있었다. 쓴 적 없는 주소 읽기, 두 버튼 동시 누름, 전원을 껐다 켠 뒤의 값은 이번 영상에 없다. DIP 레버 위치는 영상 해상도로 읽기 어려워 주소 값은 표에 적지 않았다.

## 8 FPGA-03 DAC

### 설계

ramp8이 500,000클록(10 ms)마다 값을 1 올리고, dac_parallel8이 그 값을 8비트 병렬 DAC에 쓰면서 /WR을 한 클록 LOW로 만든다. /CS와 /LDAC는 0으로 고정했다. 같은 값을 LED1~8에도 낸다. 256단계 × 10 ms = 2.56 s 주기의 톱니파가 예상이다[2].

```
always @(posedge clk_50mhz) begin
  if(rst_p) begin count<=0; value<=0; end
  else if(update) begin count<=0; value<=value+1'b1; end
  else count<=count+1;
end
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/95b1fc7/lab6/lab6_03_dac/src/ramp8.v#L9-L13

### 보드 관찰




그림 8. FPGA-03 보드 사진. 입력 조건: 손대지 않고 촬영(DIP_SW1 = 0, 채널 A). 확인한 동작: LED1~8이 이진수로 증가하고 255 다음 0으로 돌아간다. 왼쪽 아래 전압계 막대는 값이 작을 때만 채워지는 중이다.

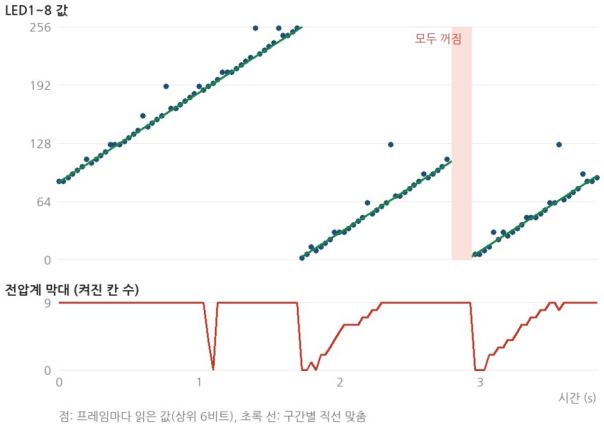



그림 9. FPGA-03 측정. 위는 프레임마다 읽은 LED 값과 구간별 직선, 아래는 전압계 막대에서 켜진 칸 수.

| 구간 (s) | 기울기 (/s) | 한 단계 | 사건 |
|---|---|---|---|
| 0~1.73 | 100.2 | 9.98 ms | 1.71 s에 255 → 0 (계산) |
| 1.73~2.80 | 98.8 | 10.13 ms | 1.70 s에 0 (계산) |
| 2.93~3.9 | 99.7 | 10.04 ms | 2.91 s에 0 (계산) |

측정값:
https://github.com/tae2yi/UOS_ece2/blob/95b1fc7/lab6/report/final/data/dac_led_bar.csv

LED7·8(하위 2비트)은 40·20 ms마다 바뀌어 한 프레임 안에서 섞이므로 상위 6비트로 값을 정하고 2를 더했다. 세 구간의 기울기는 초당 98.8~100.2로, 한 단계 10 ms 설계와 1.3 % 안에서 같다. LED1(MSB)은 0.47 s에 켜지고 1.73 s에 꺼져 1.27 s 동안 켜져 있었다(설계 1.28 s). 첫 구간의 직선이 256이 되는 1.71 s와 둘째 구간의 직선이 0이 되는 1.70 s가 같아, 255 다음 바로 0으로 돌아간 것도 확인된다.

2.80~2.93 s에는 LED가 모두 꺼졌다가 0부터 다시 늘었다. 값이 96 근처에서 갑자기 0이 되었으므로 255 → 0 순환이 아니라, 화면 밖에서 리셋(SW1, K4)이 눌린 것으로 본다. 셋째 구간의 직선이 0이 되는 2.91 s가 꺼짐이 끝난 시각과 같다.

### 전압계 막대와 최대 전압

보드의 전압계 막대는 DAC 출력을 LED 칸 수로 보여 준다. 값이 0으로 돌아간 직후 꺼졌다가 약 0.5 s(값 약 50) 동안 한 칸씩 채워지고, 그 뒤로는 계속 가득 차 있었다. 막대의 최대 눈금이 DAC 출력 범위의 약 1/5이라 그 위의 전압은 구분되지 않는다. 그래서 예비보고서에서 기록하려던 최대 전압(기준 전압 × 255/256)은 얻지 못했다. 1.10 s 한 프레임에서 막대가 모두 꺼진 것은 LED 값이 정상이라 원인을 정하지 못했다.

### 예상과의 비교

「LED1~8 이진수 카운트」, 「LED1이 1.28 s마다 바뀜」, 「0에서 2.56 s 동안 올라 다시 0」은 측정과 같다. LED8은 예상대로 항상 켜진 듯 보였다. 출력 전압 자체는 막대가 0에서부터 올라가는 모양으로 톱니파의 앞부분만 확인했고, 멀티미터나 오실로스코프 값은 없다. DIP_SW1로 채널 B를 고른 경우도 이번 영상에 없다.

## 9 토론

### 예상과 실제의 비교

| 예상 (예비보고서) | 실제 | 판정 |
|---|---|---|
| 과제5 '1' 'A' 'a' → 31 41 61 | 0x31 0x41 0x61 | 일치 |
| 과제6 결과 0, FPGA 포트에 문자 | 1 a A 수신 | 일치 |
| 과제7 '1' 켬, '0' 끔 | 2.30 s 켬, 4.20 s 끔 | 일치 |
| 과제7 LF 등 그 밖: 유지 | LF 뒤 유지 | 일치 |
| 과제7 '2' 반전 | 영상에 없음 | 미확인 |
| 서보 리셋 뒤 가운데 | 약 90° | 일치 |
| 서보 한 번에 한 단계 | 약 9°씩 | 일치 |
| 서보 1·2 ms 끝에서 멈춤 | 45°·135°에서 멈춤 | 일치 |
| SRAM 읽기 때만 쓴 값 | 0011, 0010 | 일치 |
| DAC 10 ms마다 +1 | 9.98~10.13 ms | 일치 |
| DAC 최대 전압 기록 | 막대 포화 | 미확인 |
| PWM·/WR 파형 | 오실로스코프 없음 | 미확인 |

### 두 보드를 잇는 직렬 통신

과제6·7은 MCU와 FPGA가 클록 선 없이 같은 9600 bps 약속만으로 바이트를 주고받은 실험이다. MCU는 100 MHz를 10417로, FPGA는 50 MHz를 5208로 나눠 두 쪽 속도 차이가 0.01 %이고, 받는 쪽이 각 비트의 가운데에서 읽으므로 여유가 크다. 실제로 영상의 모든 문자와 LF가 깨지지 않고 도착했다.

### 오차와 한계

- 영상의 시간 분해능은 0.033 s다. DAC의 기울기는 20~50프레임에 직선을 맞춰 구했다.
- 서보 각도는 비스듬히 찍은 장면에서 눈금을 읽어 약 ±5°의 오차가 있다.
- 서보 PWM, DAC /WR 파형과 출력 전압을 계측기로 재지 않았다.

### 개선 방법

- 서보: 오실로스코프로 AA22의 20 ms 주기와 1.0·1.5·2.0 ms 폭을 함께 기록한다.
- DAC: 멀티미터로 CH0의 최솟값·최댓값을 재어 기준 전압과 1 LSB 크기를 구한다.
- 과제7: '2'를 여러 번 보내 반전을 확인하고, 줄 끝 없음으로 바꿔 LED2~8에 문자가 남는지 본다.

## 10 결론

STM32_2 과제5에서 MCU가 받은 문자를 0x31·0x61·0x41로 출력했고, 과제6에서 MCU → FPGA → PC로 1·a·A가 그대로 전달되었다. 과제7에서 FPGA LED1은 '1'에 켜지고 '0'에 꺼졌으며 LF 같은 다른 바이트에는 유지되었다. 한 프레임 안에 문자와 LF가 함께 LED2~8에 찍히는 것까지 관찰해 MCU가 바이트를 하나씩 보내는 과정을 확인했다.

FPGA 서보는 타이밍을 맞춘 대비안으로 버튼마다 약 9°씩, 45°~135°에서 움직였다. SRAM은 읽기 때만 저장된 값 3과 2를 LED에 보여 양방향 버스 전환이 동작했다. DAC는 초당 약 100씩 증가해 10 ms 갱신과 2.56 s 주기를 확인했으나, 보드 전압계가 포화되어 최대 전압은 재지 못했다.

## 11 제출 자료

| 항목 | 위치 |
|---|---|
| 저장소 | https://github.com/tae2yi/UOS_ece2 |
| 커밋 | 95b1fc7cbd653d63248a27790b74d1a2d77bb8a1 |
| MCU 코드 | lab6/stm32_lab/Core/Src/ece2_474_routine.c |
| FPGA 설계 | lab6/lab6_01_servo ~ lab6_04_uart_led |
| 서보 대비안 | lab6/lab6_01_servo/variants/servo_pwm_mul.v |
| 영상·설명 | lab6/report/final/videos/README.md |
| 측정값 | lab6/report/final/data/*.csv |

소스: https://github.com/tae2yi/UOS_ece2/tree/95b1fc7/lab6

영상: https://github.com/tae2yi/UOS_ece2/tree/master/lab6/report/final/videos

## 참고 문헌


[1] STMicroelectronics, STM32CubeG4 MCU Firmware Package (stm32g4xx_hal_uart.c). https://github.com/STMicroelectronics/STM32CubeG4

[2] Sarah L. Harris and David Harris, Digital Design and Computer Architecture: ARM Edition, Morgan Kaufmann, 2015. ISBN 978-0-12-800056-4.

[3] M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th ed., Pearson, 2018. ISBN 978-0-13-454989-7.